CHAPTER 9 · INFRASTRUCTURE AS CODE

Managing Terraform Backends

Where Terraform state lives, how teams share it safely, and how to move from local files to locked, versioned cloud storage.

Local vs. Remote

AWS · Azure · GCS

State Locking

Migration Walkthrough

AGENDA

What We'll Cover

Local vs. Remote Backends

Why teams outgrow the default local state file

Configuring Remote Backends

Hands-on setup for AWS S3, Azure Blob, and Google Cloud Storage

State Locking with DynamoDB

Preventing corruption when multiple engineers apply at once

Migration Scenario

Step-by-step: moving existing local state into S3 safely

Chapter Summary

Key takeaways and a quick recap checklist

9.1 THE CORE TRADE-OFF

Local Backend vs. Remote Backend

Feature	Local Backend	Remote Backend (S3 / Azure / GCS)
Storage location	On the local machine (terraform.tfstate)	Cloud storage — S3, Azure Blob, GCS, Terraform Cloud
Collaboration	Hard to share across a team	Multiple users can access the same state
Security	Risk of loss if the machine fails	Cloud security, encryption, IAM control
State locking	❌ Not supported	✔️ Supported (e.g., DynamoDB for S3)
Versioning	❌ No automatic versioning	✔️ Cloud storage versioning

Local Backend (Default)

If no backend is configured, Terraform stores state in the project directory:

```
terraform/
├── main.tf
└── terraform.tfstate
```

Problems with the Local Backend

- Difficult to collaborate with multiple team members
- No state locking — risk of corrupting state
- If the local machine crashes, state is lost

Remote Backend: AWS S3

1 Create the S3 bucket

```
$ aws s3api create-bucket \
    --bucket my-terraform-state-bucket \
    --region us-east-1
```

2 Enable bucket versioning

```
$ aws s3api put-bucket-versioning \
    --bucket my-terraform-state-bucket \
    --versioning-configuration Status=Enabled
```

Step 3 · Configure backend.tf

```
terraform {
  backend "s3" {
    bucket  = "my-terraform-state-bucket"
    key     = "terraform.tfstate"
    region  = "us-east-1"
    encrypt = true
  }
}
```

Running terraform init stores state in S3 from now on.

Remote Backend: Azure Blob Storage

Step 1 · Create a storage account & container

```
$ az storage account create \
    --name terraformstate123 \
    --resource-group myResourceGroup \
    --location eastus --sku Standard_LRS
$ az storage container create \
    --name terraformstate \
    --account-name terraformstate123
```

Running terraform init will configure the remote backend automatically.

Step 2 · Configure backend.tf

```
terraform {
  backend "azurerm" {
    resource_group_name  = "myResourceGroup"
    storage_account_name = "terraformstate123"
    container_name       = "terraformstate"
    key                  = "terraform.tfstate"
  }
}
```

Remote Backend: Google Cloud Storage

Step 1 · Create a GCS bucket

```
$ gsutil mb -l us-east1 \
    gs://my-terraform-state-bucket/
```

Step 2 · Configure backend.tf

```
terraform {
  backend "gcs" {
    bucket = "my-terraform-state-bucket"
    prefix = "terraform/state"
  }
```

Running terraform init will store the state in Google Cloud Storage — the prefix acts like a folder path inside the bucket, useful for separating environments (dev/, staging/, prod/).

State Locking with DynamoDB

If multiple users apply Terraform at the same time, it can cause state corruption. Locking allows only one user to modify state at a time.

Step 1 · Create a DynamoDB table for locking

```
$ aws dynamodb create-table \
    --table-name terraform-lock \
    --attribute-definitions \
        AttributeName=LockID,AttributeType=S \
    --key-schema \
        AttributeName=LockID,KeyType=HASH \
    --billing-mode PAY_PER_REQUEST
```

Step 2 · Enable locking in backend.tf

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-lock"
  }
}
```

9.5 HANDS-ON SCENARIO

Migrating Local State to S3 + DynamoDB

1 Initialize locally

```
terraform init
terraform apply -
auto-approve
```

Creates infrastructure; state saved locally.

2 Add backend.tf

```
backend "s3" {
  ...
  dynamodb_table =
"terraform-lock"
}
```

Declare the S3 + DynamoDB remote backend.

3 Migrate state

```
terraform init -
migrate-state
```

Terraform detects the change and migrates state.

4 Verify in S3

```
aws s3 ls s3://my-
terraform-state-
bucket
```

Confirm terraform.tfstate now lives in the bucket.

5 Apply remotely

```
terraform apply -
auto-approve
```

State is now stored and locked remotely.

9.6 CHAPTER RECAP

Key Takeaways

- Local vs. remote backends — remote storage adds collaboration, security, and versioning that local state can't provide.
- Storing state in AWS S3, Azure Blob, and Google Cloud Storage — each configured through a simple backend.tf block.
- State locking with DynamoDB — prevents two engineers from corrupting state by applying at the same time.
- Migration scenario — terraform init -migrate-state safely moves existing local state into a remote, locked backend.